

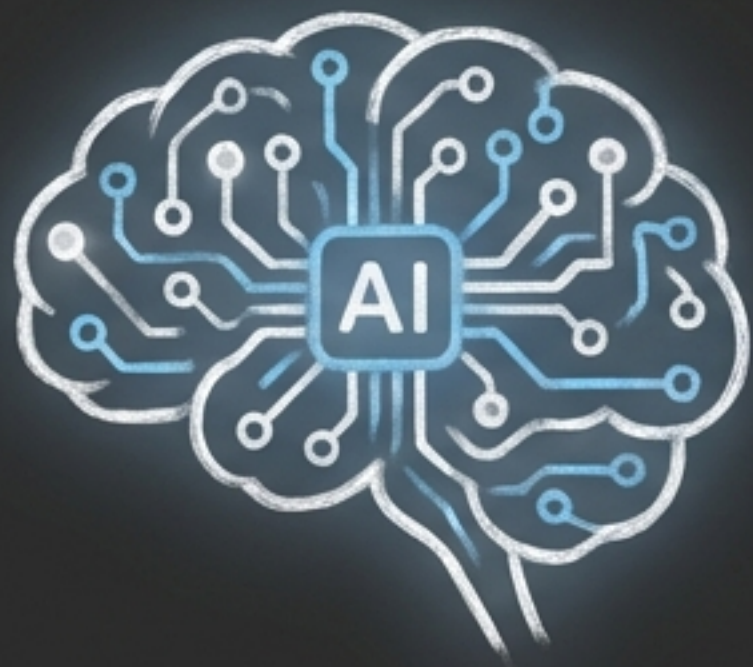
如何从零开始，亲手搭建一个RAG架构

一份写给程序员的“真理引擎”建造指南



我们面临的挑战：大语言模型的“幻觉”

请帮我写一个程序



当然，我在网上搜到了这些内容...
[一段虚构的代码]

[虚假引用来源 A]

[不存在的函数名]

幻觉

[错误的统计数据]

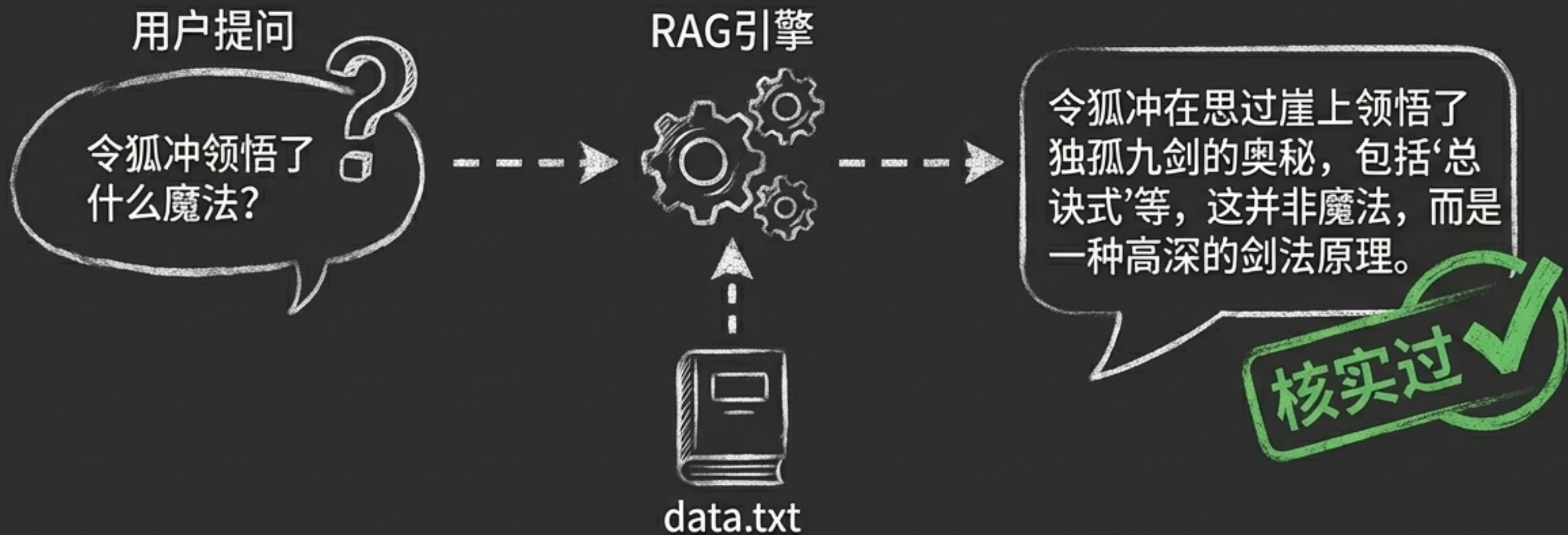
即大语言模型在回答时，
可能会一本正经地‘发明’事实。

[错误的统计数据]

[拼凑的逻辑片段]

我们的解决方案：检索增强生成（RAG）

RAG = 给大模型一本可以随时查阅的“开卷考试参考书”。



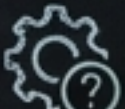
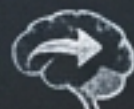
RAG架构通过从我们自己的知识库中检索相关信息，来为大模型的回答提供事实依据，从而大幅减少幻觉。

我们的建造计划

第一章：蓝图概览

- 理解RAG的两大核心流程：索引与检索。

第二章：动手建造（代码实战）

- 步骤1：准备“建材”（数据分块 - ``chunk.py``）
- 步骤2：建立“知识索引”（向量化与存储 - ``embed.py``）
- 步骤3：启动“搜索引擎”（查询与召回 - ``embed.py``）
- 步骤4：组装“生成引擎”（构建提示词与生成 - ``embed.py``）

第三章：竣工验收

- 运行我们的RAG系统并检验成果。
- 总结与展望。

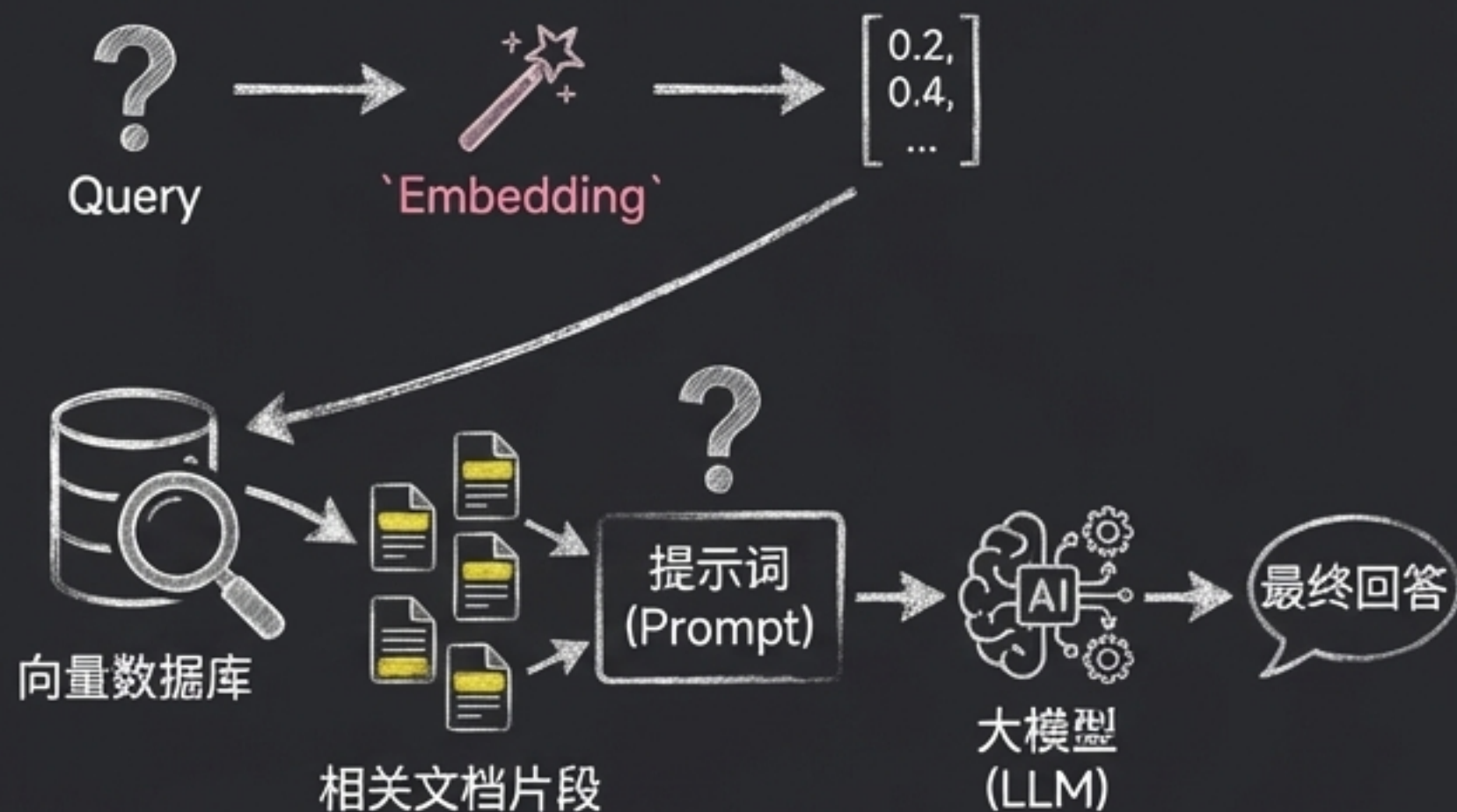
RAG架构核心蓝图：两大流程

流程一：知识索引 (Indexing Flow)



将我们的知识文档预处理、向量化，并存入专门的向量数据库中，方便后续的语义搜索。

流程二：检索与生成 (Retrieval & Generation Flow)



当用户提问时，我们同样将其向量化，在数据库中找到最相关的信息，并将其作为上下文，辅助大模型生成精准的回答。

建造步骤 1: 准备“建材” – 数据分块 (chunk.py)

目标: “将原始文档 (data.txt) 切割成大小适中、包含上下文的‘知识砖块’, 便于后续的向量化和检索。”

原材料 (Raw Material)

data.txt

```
# 第一章: 风陵渡的奇缘...  
剑魔独孤求败的传承者, 风清扬, 在华山之巅...  
...  
  
# 第二章: 史莱姆的生存法则...  
作为一只史莱姆, 令狐冲发现...
```

输出预览 (Output Preview)

```
> python chunk.py  
# 第一章: 风陵渡的奇缘...  
剑魔独孤求败的传承者, 风清扬, 在华山之巅...  
-----  
# 第二章: 史莱姆的生存法则...  
作为一只史莱姆, 令狐冲发现...
```

工具 (chunk.py) – 数据分块脚本

```
def chunk_data(data_path):  
    """读取数据并将其分块。"""  
    with open(data_path, 'r', encoding='utf-8') as f:  
        content = f.read()  
  
    # 我们使用空行作为分隔符, 将文章切分成段落。  
    chunks = content.split('\n\n')  
  
    final_chunks = []  
    current_chapter = "无章节"  
    for c in chunks:  
        if c.strip():  
            if c.startswith('#'):  
                current_chapter = c.strip()  
            else:  
                # 确保每个知识块都带有其所属的章节标题作为上下文  
                final_chunks.append(current_chapter + "\n" + c.strip())  
    return final_chunks  
  
if __name__ == "__main__":  
    chunks = chunk_data("data.txt")  
    for i, chunk in enumerate(chunks[:2]): # 预览前两个块  
        print(chunk)  
        print("-----")
```

我们使用空行作为分隔符, 将文章切分成段落。

这段逻辑非常关键: 它能识别出章节标题, 并确保每个知识块都带有其所属的章节标题作为上下文。这极大地提升了后续检索的准确性。

建造步骤 2: 建立“知识索引” – 向量化与存储

Embedding



什么是Embedding?

“简单来说，就是用一个‘魔法函数’，将文本（语义）转换成一个由数字组成的向量（数学表示）。语义相近的文本，其向量在空间中的距离也相近。”

Vector DB



向量数据库
ChromaDB

什么是向量数据库?

“一个专门用来存储和高效查询这些‘文本向量’的特殊数据库。我们选用`ChromaDB`。”

代码展示 (embed.py – Part 1: 设置)

```
# -*- coding: utf-8 -*-  
import chromadb  
from google import genai  
import os
```

引入我们的向量数据库。

引入提供Embedding和大语言模型的库。

```
# 配置Embedding模型
```

```
EMBEDDING_MODEL = "gemini-embedding-exp-03-07"
```

定义我们选择的Embedding模型。

```
# 初始化ChromaDB客户端
```

```
chromadb_client = chromadb.PersistentClient(path="./chroma_db")
```

初始化ChromaDB客户端，让数据持久化存储。

```
# 获取或创建集合
```

```
chromadb_collection = chromadb_client.get_or_create_collection(name="linghuchong")
```

获取或创建一个名为‘linghuchong’的集合（相当于数据库中的一张表）。

```
# (Part 2: 向量化与存储逻辑将在下一步展示)
```

核心代码解析 1: `embed` 函数

这个函数是我们的‘向量化转换器’。它接收文本，并根据任务类型（是存储文档还是查询问题）调用 Google GenAI API，返回一个向量。

```
# embed.py
```

```
def embed(text: str, store: bool) -> list[float]:
```

函数签名清晰地定义了输入（文本、是否存储）和输出（浮点数列表，即向量）。

```
    result = google_client.models.embed_content(
```

```
        model=EMBEDDING_MODEL,
```

```
        contents=text,
```

```
        config={
```

```
            "task_type": "RETRIEVAL_DOCUMENT" if store else "RETRIEVAL_QUERY"
        })
```

一个非常重要的参数。我们明确告诉模型，这次向量化是为了‘存储文档’还是为了‘查询’。这会影响向量生成的质量，让查询向量和文档向量更好地匹配。

```
    assert result.embeddings
```

```
    assert result.embeddings[0].values
```

```
    return result.embeddings[0].values
```

从API返回结果中提取出核心的向量数据。

核心代码解析 2: `create\_db` 函数

这是我们的‘知识入库’流程。它遍历所有文本块，逐一调用`embed`函数进行向量化，然后将原文、向量和ID一起存入ChromaDB。

```
# embed.py

def create_db():
    # 导入我们的文本切割模块
    import chunk

    print("开始知识入库...")
    # 遍历所有文本块
    for idx, c in enumerate(chunk.get_chunks()): ← 循环处理每一个从`chunk.py`获取的知识块。
        print(f"正在处理第 {idx} 个文本块...")

        # 进行向量化
        embedding = embed(c, store=True) ← 调用`embed`函数，并将`store`设为`True`，表明这是为了存储文档。

        # 存入向量数据库
        chromadb_collection.upsert(
            ids=str(idx), ← 为每个文档块分配一个唯一ID。
            documents=c, ← 存储原始文本。
            embeddings=embedding ← 存储转换后的向量。
        )
        print(f"第 {idx} 个文本块已存入。")

    # 简单的延时策略
    time.sleep(2) → 添加一个简单的延时，避免因请求频率过快而被API限流。
                    在生产环境中会有更完善的策略。
```

这是关键的存储步骤。

建造步骤 3：启动“搜索引擎” — 查询与召回

当用户提问时，这个`query\_db`函数会：1. 将问题向量化；2. 在ChromaDB中执行语义搜索；3. 返回最相似的N个知识块。

```
# embed.py
```

```
def query_db(question: str) -> list[str]:
```

```
# 1. 将问题向量化
```

```
question_embedding = embed(question, store=False)
```

同样调用`embed`函数，但这次`store`为`False`，表明这是一个查询请求。

```
# 2. 在ChromaDB中执行语义搜索
```

```
result = chromadb_collection.query(
```

```
    query_embeddings=question_embedding, ← 传入问题的向量。
```

```
    n_results=5 ←
```

指定我们希望返回最相关的5个结果。

} 执行查询操作。

```
# 3. 返回最相似的N个知识块
```

```
return result["documents"][0] ← 从查询结果中提取出原始文本文档并返回。
```

建造步骤 4：组装“生成引擎” – 运行完整流程

这是我们程序的“**总开关**”。它按顺序执行所有步骤，并将检索到的知识与原始问题组合成一个强大的提示词（Prompt），最后交给大模型生成答案。

```
# embed.py
if __name__ == '__main__':
    # 1. 定义测试问题
    question = "令狐冲领悟了什么魔法?"
    print(f"正在提问: {question}")

    # 2. 确保知识库已建立 (实际应用中, 知识库只需建立一次)
    create_db()

    # 3. 执行检索
    chunks = query_db(question)
    print(f"检索到的相关知识块: \n{chunks}")

    # 4. 构建Prompt并发送给LLM生成回答
    # 这里将检索到的上下文和问题组合成一个完整的提示词
    prompt = f"""
    请根据以下【上下文】，回答【用户的问题】。如果你在上下文中找不到答案，请直接回答“我不知道”。

    【上下文】
    {"".join(chunks)}

    【用户的问题】
    {question}
    """

    # 调用大模型
    result = google_client.models.generate_content(
        model="gemini-1.5-flash",
        contents=prompt
    )

    # 5. 打印最终回答
    print("-" * 30 + "\n最终回答: ")
    print(result.text)
```

定义我们的测试问题。

首先，确保知识库已经建立。

然后，根据问题检索相关知识块。

将最终的Prompt发送给大语言模型。

打印出最终的、有事实依据的回答。

****这就是RAG的精髓****：我们构建一个结构化的Prompt，清晰地告诉LLM：‘请根据我提供的【上下文】，回答【用户的问题】’。

第三章：竣工验收 - 启动我们的“真理引擎”

步骤1：切换到项目目录

```
D:\WORKPLACE_WQQ> cd RAG
```

步骤2：运行Python脚本

```
D:\WORKPLACE_WQQ\RAG> python embed.py
```

步骤3：查看结果

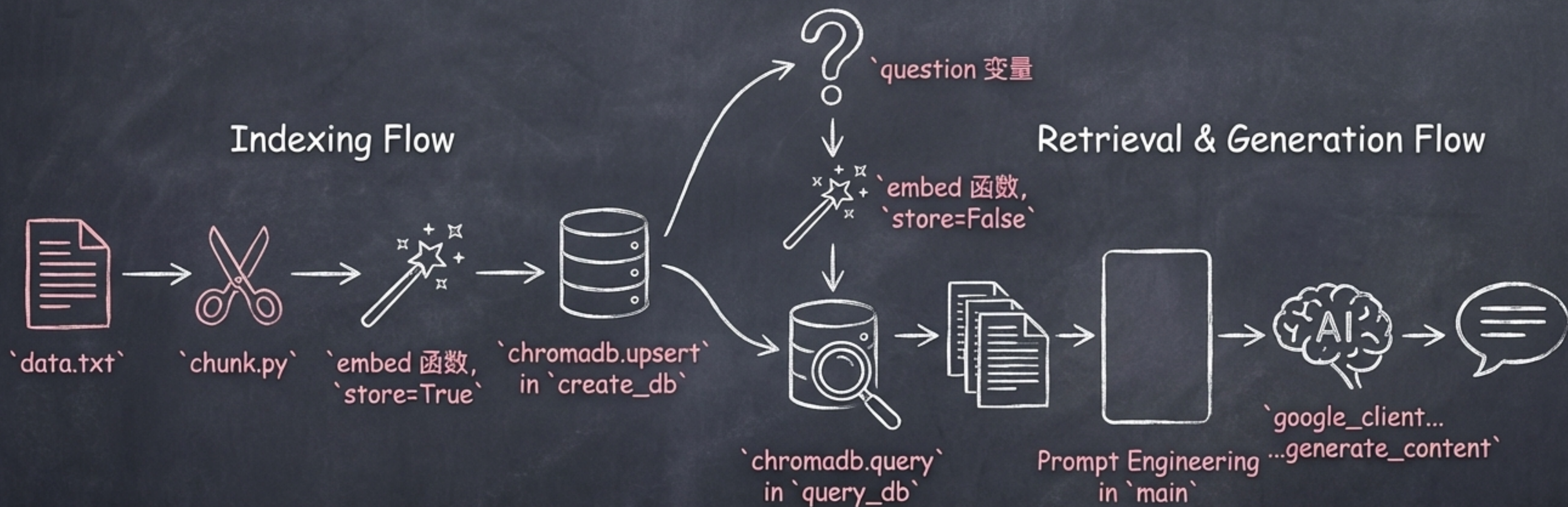
```
Process: # 第一章：风陵渡的奇缘...
```

```
Process: # 第二章：史莱姆的生存法则...
```

根据提供的上下文，令狐冲并未领悟任何“魔法”。他领悟的是一门高深的剑术，称为“独孤九剑”。这门剑术的精髓在于“无招胜有招”，讲求的是根据对手的招式变化而随机应变，并非传统意义上的魔法。

成功！我们的RAG系统没有产生幻觉，而是根据提供的上下文给出了准确的回答。

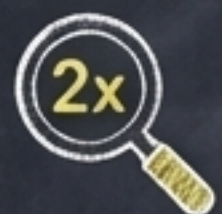
我们的成果：RAG架构全景回顾



我们从一份原始文档开始，通过分块、向量化、存储、查询和提示词工程，成功构建了一个能够利用私有知识、回答特定问题的完整RAG系统。

蓝图升级：从基础到进阶的RAG

今天我们搭建的是一个坚实的基础版RAG。在实际应用中，还有许多可以优化的方向，让你的“真理引擎”更强大：



更优的检索策略 (Advanced Retrieval)：例如使用`Re-ranking`模型对初步检索的结果进行二次排序，提高最相关文档的优先级。



更智能的分块方法 (Smarter Chunking)：不仅仅是按段落分割，可以按语义、固定大小加重叠等方式，保留更完整的上下文。



查询转换 (Query Transformation)：在某些场景下，可以先让LLM“重写”或“扩展”用户的原始问题，生成更适合向量检索的查询语句。



混合搜索 (Hybrid Search)：结合传统的关键词搜索（如BM25）和向量的语义搜索，取长补短。

掌握了今天的核心构建方法，你就拥有了探索这些前沿技术的钥匙。祝你建造愉快！